Time Synchronisation Between Klipper Motion and Trinkey Accelerometer Data

1 Purpose

The frequency response function (FRF) identification requires the commanded input acceleration and the measured output acceleration to be expressed on the same time axis. In this experiment, the input motion is generated by Klipper, while the output acceleration is measured by two BNO055 accelerometers connected to a separate Trinkey QT2040 board. Therefore, the system contains two relevant time bases:

- Klipper *print time*, which is the time base used to schedule motion on the printer MCU.
- Trinkey time, given by the timestamp t_{us} produced by the RP2040 timer on the Trinkey.

The aim of the synchronisation procedure is to map each Trinkey timestamp to Klipper print time. After this mapping, the input and output signals can be compared sample-by-sample using a common time coordinate.

2 Klipper Print Time

Klipper plans motion on the host computer, but the final step pulses are executed by the printer MCU. Internally, Klipper uses a variable called *print time* to represent the scheduled time of a motion command. For the primary MCU, print time is directly related to the MCU clock:

$$c = f_{\mathrm{mcu}} p, \tag{1}$$

where c is the MCU clock value in ticks, f_{mcu} is the MCU clock frequency, and p is Klipper print time in seconds. Therefore, print time can be interpreted as the MCU clock expressed in seconds.

When a vibration test is executed, each input motion segment has a known print time. If the first segment starts at print time p_0, and a generated segment starts at local test time τ_k, then its scheduled start time is

$$p_k = p_0 + \tau_k. \tag{2}$$

The corresponding input acceleration for that segment is known from the trajectory generation code. Therefore, the input acceleration can be stored as a piecewise constant signal in Klipper print time:

$$u(p) = a_k, \qquad p_k \le p < p_k + \Delta t_k, \tag{3}$$

where a_k is the acceleration of segment k, and Δt_k is its duration.

3 Trinkey Time

The Trinkey firmware records accelerometer samples with its own timestamp t_{us}, measured in microseconds. This timestamp is produced by the Trinkey RP2040 and is independent from the printer MCU clock. For analysis it is first converted to seconds:

$$q = 10^{-6} t_{\text{us}}. \tag{4}$$

The two BNO055 accelerometers are connected to the same Trinkey board. Therefore, both accelerometers share the same Trinkey time base. This is useful, because one time synchronisation model can be used for both the toolhead and bed accelerometer data.

4 Clock Model

Over the duration of one vibration sweep, the relation between Trinkey time and Klipper print time can be modelled as an affine function:

$$p = \alpha q + \beta, \tag{5}$$

where:

- q is Trinkey time in seconds,
- p is Klipper print time in seconds,
- α is the relative clock rate,
- β is the clock offset.

If both clocks ran at exactly the same rate, then $\alpha = 1$. In practice, α can be slightly different from one due to oscillator tolerances and clock drift. The offset β accounts for the fact that the two clocks do not start from the same zero time. This is the standard clock offset and clock skew model used in clock synchronisation methods, including sensor-network synchronisation approaches [3].

5 Synchronisation Measurements

To estimate α and β, the Klipper host sends repeated synchronisation requests to the Trinkey during the experiment. For one synchronisation request:

1. The Klipper host records its monotonic time before sending the request: $h_{0,i}$.
2. The host sends a synchronisation command to the Trinkey.
3. The Trinkey immediately replies with its current timestamp q_i.
4. The host records its monotonic time after receiving the reply: $h_{1,i}$.

The host time corresponding to the Trinkey reply is estimated by the midpoint:

$$h_i = \frac{h_{0,i} + h_{1,i}}{2}. \tag{6}$$

The round-trip time is

$$r_i = h_{1,i} - h_{0,i}. \tag{7}$$

The midpoint approximation assumes that the communication delay is approximately symmetric. This is not perfectly true for USB serial communication, but repeated measurements allow high-latency outliers to be rejected. This follows the same general principle as two-way clock synchronisation methods such as Cristian's algorithm and the Network Time Protocol, where round-trip delay is used to estimate and qualify the clock offset measurement [1, 2].

The host monotonic time h_i is then converted to Klipper print time using Klipper's clock synchronisation estimate:

$$p_i = \text{estimated_print_time}(h_i). \tag{8}$$

Each synchronisation request therefore gives one calibration pair:

$$(q_i, p_i). \tag{9}$$

6 Least-Squares Clock Fit

The parameters α and β are estimated by fitting the model in Equation (5) to all accepted synchronisation pairs. This is an ordinary linear least-squares problem with an intercept term [4]. The measurements can be written in matrix form as

$$p = X\theta + e, \tag{10}$$

where

$$p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_N \end{bmatrix}, \qquad X = \begin{bmatrix} 1 & q_1 \\ 1 & q_2 \\ \vdots & \vdots \\ 1 & q_N \end{bmatrix}, \qquad \theta = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}. \tag{11}$$

Here, p is the vector of Klipper print times, q_i are the Trinkey timestamps converted to seconds, and e is the timing error in the synchronisation measurements. The first column of X is equal to one because the model includes the offset β. The least-squares estimate is obtained by minimizing the squared residual norm:

$$\hat{\theta} = \arg\min_{\theta} \|p - X\theta\|_2^2 . \tag{12}$$

The closed-form normal-equation solution is

$$\hat{\theta} = (X^T X)^{-1} X^T p, \tag{13}$$

provided that $X^T X$ is non-singular. In implementation, the inverse should not be formed explicitly. MATLAB's backslash operator or a QR/SVD least-squares solver should be used instead, for example $\hat{\theta} = X \backslash p$. This is numerically more stable and matches the usual implementation of least squares in system identification scripts.

After these parameters have been estimated, every accelerometer sample can be mapped to Klipper print time:

$$p_{\text{sample}} = \alpha \left(10^{-6} t_{\text{us}}\right) + \beta. \tag{14}$$

7 Latency Rejection

USB and operating system scheduling can occasionally delay a synchronisation request. Such delayed requests should not be used directly, because they reduce the accuracy of the clock fit. The round-trip time r_i is used as a quality measure.

A simple rejection rule is

$$r_i \leq r_{\min} + r_{\text{tol}}, \tag{15}$$

where $r_{\min}$ is the minimum observed round-trip time and r_{tol} is a chosen tolerance, for example 1 ms. Synchronisation pairs with larger round-trip time are rejected.

An alternative is to use weighted least squares. In that case, lower-latency measurements receive a larger weight:

$$w_i = \frac{1}{r_i^2}. \tag{16}$$

This can also be written in matrix form by defining a diagonal weighting matrix

$$W = \begin{bmatrix} w_1 & 0 & \cdots & 0 \\ 0 & w_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & w_N \end{bmatrix}. \tag{17}$$

The weighted least-squares estimate is then

$$\hat{\theta} = (X^T W X)^{-1} X^T W p. \tag{18}$$

For this experiment, the simpler rejection method is usually easier to explain and verify. Weighted fitting can be added if the observed round-trip times vary significantly.

8 Alignment of Input and Output Signals

After synchronisation, both the input and output are represented in Klipper print time. The input is the commanded acceleration $u(p)$, obtained from the motion segments generated by Klipper. The output is the measured acceleration $y(p)$, obtained by converting each Trinkey sample timestamp using Equation (14).

The two signals can then be compared on the same time axis:

$$u(p) \quad \text{and} \quad y(p). \tag{19}$$

For frequency-domain identification, the steady-state part of the response is selected after the initial ramp. The complex gain at angular frequency $\omega = 2\pi f$ can be estimated by fitting sine and cosine components to the measured output:

$$y(p) \approx B \sin(\omega p) + C \cos(\omega p). \tag{20}$$

The same time base must be used for the input and output. Otherwise, a timing error Δt introduces a phase error

$$\Delta\phi = -\omega \Delta t. \tag{21}$$

In degrees this is

$$\Delta\phi_{\text{deg}} = -360 f \Delta t. \tag{22}$$

This shows why time synchronisation is important. For example, a timing error of 1 ms causes a phase error of 1.8 deg at 5 Hz, but 36 deg at 100 Hz.

9 Remaining Sources of Phase Error

The proposed synchronisation method removes the main timing uncertainty caused by USB serial latency and host-side logging. However, it does not remove all possible phase errors. The following effects may still remain:

- BNO055 internal filtering and sensor group delay.
- Small timing differences between reading the bed and toolhead accelerometers, because they are read sequentially on the same I2C bus.
- Quantisation and jitter in the generated motion segments.
- Mechanical phase lag caused by the printer structure, which is the desired quantity for FRF identification.

These effects should be considered when interpreting the measured phase. In particular, any approximately constant additional delay can be estimated and reported separately from the mechanical dynamics.

10 Summary

The synchronisation procedure maps Trinkey accelerometer timestamps onto Klipper print time using repeated sync requests and a linear clock model. The main steps are:

1. Log the commanded input acceleration using Klipper print time.
2. Log each accelerometer sample with its Trinkey timestamp.
3. Collect synchronisation pairs between Trinkey time and Klipper print time.
4. Reject high-latency synchronisation pairs using the measured round-trip time.
5. Fit the affine model $p = \alpha q + \beta$.
6. Convert all accelerometer samples to Klipper print time.
7. Use the common time base for FRF estimation.

This approach follows the same principle as Klipper's built-in accelerometer handling: sample timestamps are not based on when data arrives at the host, but on a local sensor clock that is later translated to Klipper's motion time base.

References

[1] F. Cristian, "Probabilistic clock synchronization," *Distributed Computing*, vol. 3, no. 3, pp. 146–158, 1989. doi: 10.1007/BF01784024.

[2] D. L. Mills, J. Martin, J. Burbank, and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, Internet Engineering Task Force, 2010. doi: 10.17487/RFC5905.

[3] J. Elson, L. Girod, and D. Estrin, "Fine-grained network time synchronization using reference broadcasts," in *Proceedings of the 5th Symposium on Operating Systems Design and Implementation*, pp. 147–163, 2002. doi: 10.1145/844128.844143.

[4] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed. Wiley, 2012.